

TaskFlow

Plataforma Inteligente de Gestión de Proyectos

Propuesta de Proyecto Final

1. Descripción General

TaskFlow es una aplicación web desarrollada en Python que permite a estudiantes de la Universidad Distrital Francisco José de Caldas y grupos de estudio gestionar proyectos y tareas de forma colaborativa y en tiempo real. El sistema está diseñado para que múltiples usuarios puedan crear proyectos, asignar tareas, establecer prioridades y recibir alertas automáticas cuando una tarea está próxima a vencer o ya venció.

La propuesta integra los principales temas del curso de Programación Avanzada: programación orientada a objetos, Flask, persistencia con SQLAlchemy, API REST, serialización, concurrencia y control de versiones con Git.

2. Problema y Justificación

En grupos de estudio pequeños, o estudiantes, es común que la coordinación de tareas suela hacerse por WhatsApp, excels o listas informales lo cual genera problemas como olvido de fechas límite, falta de claridad sobre responsables y dificultad para ver el estado real del proyecto.

TaskFlow resuelve esto con un sistema centralizado, accesible desde el navegador, donde cada miembro del equipo puede ver sus tareas pendientes, el estado de los proyectos y recibir notificaciones automáticas. A diferencia de otras herramientas, TaskFlow es construido desde cero aplicando los conceptos del curso, lo que permite demostrar comprensión real de cada componente técnico.

3. Objetivo General

Desarrollar una aplicación web funcional en Python y Flask que permita gestionar proyectos y tareas colaborativas, integrando programación orientada a objetos, persistencia de datos, API REST, concurrencia y buenas prácticas de ingeniería de software.

Objetivos Específicos

- Implementar un módulo de usuarios con registro, inicio de sesión y roles (administrador y miembro).
- Crear un CRUD completo para proyectos y tareas con asignación de responsables y fechas límite.
- Aplicar concurrencia mediante hilos para monitorear tareas vencidas y registrar logs del sistema.
- Diseñar una API REST con Flask-RESTful que exponga los recursos principales del sistema.
- Documentar el uso de agentes de IA durante el desarrollo y evidenciar las decisiones técnicas tomadas.

4. Alcance y Limitaciones

Alcance del proyecto

TaskFlow contempla las siguientes funcionalidades dentro del tiempo disponible (27 de mayo al 1 de junio de 2026):

- Registro e inicio de sesión de usuarios con roles básicos (administrador y miembro).
- Creación, edición y eliminación de proyectos y tareas.
- Asignación de responsables, prioridades (alta, media, baja) y fechas límite a cada tarea.
- Cambio de estados de tareas: pendiente, en progreso, completada.
- Notificaciones automáticas generadas por un hilo en segundo plano cuando una tarea vence.
- API REST con endpoints para consultar y modificar tareas y proyectos en formato JSON.
- Exportación básica del estado del proyecto a un archivo JSON o CSV.

Limitaciones

Las siguientes funcionalidades quedan como propuesta, considerando que el proyecto se desarrolla en poco tiempo:

- No se implementará chat en tiempo real (Flask-SocketIO queda como mejora futura).
- No habrá envío real de correos electrónicos; las notificaciones se registran internamente en la base de datos.
- No se realizará despliegue en servidor externo; el sistema corre localmente.
- No se desarrollará una aplicación móvil ni interfaz nativa; el acceso es exclusivamente por navegador web.
- El sistema no soporta múltiples idiomas ni accesibilidad avanzada en esta versión.

5. Tecnologías a Utilizar

Tecnología	Uso en el proyecto
Python	Lenguaje principal de desarrollo
Flask	Backend web, manejo de rutas y vistas
Flask-RESTful	Construcción de la API REST
SQLAlchemy	ORM para manejo de la base de datos
SQLite	Base de datos relacional liviana
Threading	Concurrencia: hilos en segundo plano
JSON / CSV	Serialización y exportación de datos
Git & GitHub	Control de versiones y colaboración

6. Funcionalidades Principales

Módulo de Usuarios

- Registro e inicio de sesión con validación de datos.
- Roles: administrador (gestiona proyectos y usuarios) y miembro (gestiona sus tareas).

Módulo de Proyectos

- Crear, editar y eliminar proyectos.
- Compartir proyectos con otros usuarios registrados.
- Ver el estado general de avance del proyecto.

Módulo de Tareas

- Crear tareas dentro de un proyecto con nombre, descripción, prioridad y fecha límite.
- Asignar una tarea a un miembro del equipo.
- Cambiar el estado de la tarea: pendiente → en progreso → completada.

Sistema de Notificaciones y Logs

- Un hilo en segundo plano revisa periódicamente si hay tareas vencidas y genera alertas.
- Un segundo hilo registra en un archivo de log las acciones del sistema (creación, edición, cambios de estado).

API REST

- Endpoints para listar, crear, actualizar y eliminar tareas y proyectos.
- Las respuestas se devuelven en formato JSON.

7. Aplicación de los Temas del Curso

Tema del curso	Aplicación en TaskFlow
Programación Orientada a Objetos	Clases: Usuario, Proyecto, Tarea, Notificacion, LogEntry
Flask	Backend web con rutas, formularios y vistas HTML
Persistencia / SQLite / SQLAlchemy	Modelos relacionales: Usuario tiene Proyectos, Proyecto tiene Tareas
CRUD	Crear, consultar, editar y eliminar tareas y proyectos
API REST	Endpoints /api/tareas, /api/proyectos con métodos GET, POST, PUT, DELETE
Serialización	Exportación de tareas a JSON y CSV; carga de configuración desde archivo
Archivos	Lectura/escritura de logs en .txt y exportación de reportes en .csv
Concurrencia	Dos hilos: uno monitorea tareas vencidas, otro escribe logs del sistema
Patrones de diseño	MVC con Flask; patrón Repositorio para acceso a datos; posible uso de Observador para notificaciones
Principios SOLID	Separación de responsabilidades en módulos: models, routes, services, api
Git & GitHub	Ramas main/develop/feature/*, commits descriptivos, Pull Requests
Agentes de IA	Se documentará el uso de Claude/ChatGPT en diseño, código y depuración

8. Arquitectura del Sistema

taskflow/

├─ app/

| ├─ models/ # Clases: Usuario, Proyecto, Tarea, Notificación

| ├─ routes/ # Rutas Flask (vistas web)

| └─ services/ # Lógica de negocio y concurrencia

```

|   ├── api/          # Endpoints REST con Flask-RESTful
|   ├── templates/    # Vistas HTML con Bootstrap
|   └── static/        # CSS, JS, imágenes
|── data/
|   ├── input/        # Archivos de configuración JSON
|   └── output/        # Exportaciones CSV y logs
|── documentation/
|   ├── project_report.pdf # Informe del proyecto
|   └── evidence/        # Evidencias y documentación adicional
|── tests/             # Pruebas unitarias y de integración
|── requirements.txt    # Dependencias de Python
|── README.md          # Documentación principal
|── run.py             # Punto de entrada de la aplicación

```

Patrón de Diseño Principal: MVC + Repositorio

El sistema sigue el patrón MVC (Modelo-Vista-Controlador) que Flask facilita naturalmente. Adicionalmente, se aplica el patrón Repositorio para aislar el acceso a la base de datos de la lógica de negocio, lo que reduce el acoplamiento y facilita el mantenimiento del código (principio SOLID de responsabilidad única).

9. Componente de Concurrency

El sistema ejecutará tres hilos en segundo plano usando el módulo threading de Python:

Hilo	Función	Frecuencia
monitor_tareas	Revisa si alguna tarea superó su fecha límite y genera una notificación	Cada 60 segundos
registro_logs	Escribe en un archivo .txt los eventos del sistema (creaciones, cambios de estado)	En tiempo real
limpieza_sesiones	Elimina sesiones de usuario inactivas de la base de datos	Cada 10 minutos

10. Modelo de Datos Simplificado

Las siguientes entidades conforman la base de datos del sistema:

Entidad	Atributos principales	Relación
Usuario	id, nombre, email, contraseña, rol	Tiene muchos Proyectos y Tareas
Proyecto	id, nombre, descripción, fecha_creacion, id_usuario	Tiene muchas Tareas
Tarea	id, titulo, descripcion, prioridad, estado, fecha_limite, id_proyecto, id_responsable	Pertenece a Proyecto y Usuario
Notificacion	id, mensaje, fecha, leida, id_usuario, id_tarea	Pertenece a Usuario y Tarea
LogEntry	id, accion, descripcion, fecha, id_usuario	Registro de actividad del sistema

11. Cronograma de Trabajo

Fecha	Actividad
Miércoles 27 de mayo	Presentación de la propuesta. Definición de arquitectura y modelo de datos.
Jueves 28 de mayo	Exposición oral de la propuesta. Inicio de implementación: modelos y base de datos.
Viernes 29 de mayo	Desarrollo de rutas Flask, vistas HTML y lógica CRUD.
Sábado 30 de mayo	Implementación de la API REST y serialización (JSON/CSV).
Domingo 31 de mayo	Concurrencia (hilos), pruebas, corrección de errores.
Lunes 1 de junio	Entrega final: documentación, README.md, informe y sustentación.

12. Uso de Agentes de Inteligencia Artificial

El grupo utilizará herramientas de IA (Claude de Anthropic y/o ChatGPT) como apoyo en las siguientes etapas del desarrollo. El uso quedará completamente documentado en el archivo `documentation/project_report.pdf`, incluyendo los prompts utilizados, las respuestas aceptadas, las corregidas y las decisiones finales tomadas de forma autónoma por el grupo.

Etapas del proyecto	Uso previsto de la IA
Análisis y diseño	Apoyo en la definición de casos de uso y modelo entidad-relación
Código inicial	Generación de estructura base de modelos SQLAlchemy y rutas Flask
Depuración	Identificación de errores de lógica en hilos y rutas REST
Documentación	Apoyo en redacción del README.md y comentarios en el código
Pruebas	Sugerencias de casos de prueba para CRUD y endpoints de la API

El grupo es responsable de entender, verificar y defender cada fragmento de código entregado, independientemente de si fue generado con apoyo de IA.

13. Conclusión

TaskFlow es una propuesta realista, alcanzable en el tiempo disponible y diseñada para evidenciar todos los temas del curso de Programación Avanzada. El proyecto resuelve un problema concreto, aplica buenas prácticas en el desarrollo de aplicaciones y sirve como base para un posibles desarrollos

Los temas del curso no se aplicarán de forma aislada: cada componente técnico está integrado dentro de una solución funcional, lo que permite defender cada decisión de diseño durante la sustentación.